

Esercizio A1 – Complessità – Punti ≥ 8

- **Algoritmo A:**

$$T_A(n) = \begin{cases} \Theta(1) & \text{se } n \leq 1, \\ 5T_A(n/2) + cn & \text{se } n > 1. \end{cases}$$

Applichiamo il teorema delle ricorrenze lineari con partizione bilanciata: $a = 5, b = 2, \beta = 1$; si ottiene $\alpha = \log_2 5 > 2 > \beta$. Si applica quindi il caso 1 e la complessità è $\Theta(n^{\log_2 5} \approx \Theta(n^{2.32}))$.

- **Algoritmo B:**

$$T_B(n) = \begin{cases} \Theta(1) & \text{se } n \leq 1, \\ 3T_B(n-1) + c & \text{se } n > 1. \end{cases}$$

Applichiamo il teorema delle ricorrenze lineari di ordine costante: $a = 3, \beta = 0$. La complessità è quindi $\Theta(3^n)$.

- **Algoritmo C:**

$$T_C(n) = \begin{cases} \Theta(1) & \text{se } n \leq 1, \\ 9T_C(n/3) + cn^2 & \text{se } n > 1. \end{cases}$$

Applichiamo il teorema delle ricorrenze lineari con partizione bilanciata: $a = 16, b = 4, \beta = 2$; si ottiene $\alpha = \log_4 16 = 2 = \beta$. Si applica quindi il caso 2 e la complessità è $\Theta(n^2 \log n)$.

Fra i tre algoritmi, il terzo è preferibile perchè la sua complessità è inferiore.

Esercizio A2 – Tassografia – Punti ≥ 10

L'algoritmo si basa su una visita in profondità (DFS) con classificazione degli archi. Usiamo la procedura `hascycle()` già vista a lezione. Possono darsi due casi:

- **Il grafo contiene cicli:** In questo caso, classifichiamo G come *grafo generale non aciclico*.
- **Il grafo non contiene cicli:** Allora G è un DAG. Per stabilire se è anche un albero radicato orientato, calcoliamo l'in-degree di ogni nodo. Se tutti i nodi hanno in-degree 1 tranne un nodo con zero, allora è un albero radicato orientato. Altrimenti confermiamo che il grafo è un DAG e non è un albero.

Sia `hasCycle()` che valutare gli in-degree richiedono $O(m + n)$.

La procedura assume questa forma:

```
int categorizeGraph(Graph G)
{
    int n = G.size()
    int m = 0
    if hasCycle(G) then
        return 3                                % Grafo ciclico, quindi non-dag
    else
        int[] in = new int[1...n] = {0}          % In-degree
        foreach u in G.V() do
            foreach v in G.adj(u) do
                in[v] += 1
        int count0 = 0
        for u in G.V() do
            if in[u] == 0 then
                if count0 > 0 then
                    return 2                    % C'è più di un radice, DAG
                else if in[u] > 1 then
                    return 2                    % Più di un arco entrante, DAG
            count0++
        return 1                                % Albero radicato orientato
}
```

Esercizio A3 – Scendere nell'albero – Punti ≥ 12

Una soluzione banale consiste nel sondare tutti i nodi dell'albero e verificare, per ciascuno, se soddisfa la proprietà di minimo locale. Appena si trova un nodo che lo è, lo si restituisce. Questo approccio ha complessità $O(n)$, poiché nel caso pessimo bisogna visitare una porzione lineare dell'albero (in media almeno metà dei nodi).

Possiamo però ragionare in maniera più efficiente. Consideriamo la radice: se il suo valore è più piccolo di quello dei suoi figli, allora è già un minimo locale e l'algoritmo termina. Altrimenti, almeno uno dei figli ha valore minore della radice; scegliamo il figlio con valore più basso e scendiamo lungo quel ramo.

Se il nodo corrente è più piccolo dei suoi vicini, lo restituiamo come minimo locale; in caso contrario, uno dei suoi figli ha valore minore, e quindi possiamo proseguire verso quel figlio.

Iterando questo procedimento, si scende lungo un cammino radice–foglia: ad ogni passo si sceglie il figlio con valore più piccolo. Ci sono due possibilità:

- si incontra un nodo interno che è già un minimo locale, e l'algoritmo si arresta;
- si arriva a una foglia. In questo caso, la foglia è necessariamente un minimo locale: infatti il suo valore è minore di quello del genitore (da cui è stata scelta come figlio più piccolo), e non ha altri vicini.

Questo ragionamento garantisce che l'algoritmo trovi sempre un minimo locale lungo il cammino seguito, senza dover visitare l'intero albero.

Il codice è molto semplice:

TREE localMinimum(TREE T)

```
if T.left == nil and T.right == nil then
    return T                                % Se siamo arrivati a una foglia, è un minimo locale
else if T.val < T.left.val and T.val < T.right.val then
    return T                                % Le proprietà di minimo locale sono verificate in questo nodo
else if T.left.val < T.val then
    return localMinimum(T.left)             % Il valore a sinistra è più piccolo, andiamo a sinistra
else
    return localMinimum(T.right)           % Il valore a destra è più piccolo, andiamo a destra
```

Poiché stiamo operando su un albero perfetto, il percorso radice–foglia ha lunghezza $\Theta(\log n)$ e l'algoritmo ha complessità $O(\log n)$ (in quanto si può fermare prima).

Esercizio B1 – Mensa – Punti ≥ 6

Idea generale. Il problema si scompone in tre sottoproblemi indipendenti, uno per ciascuna categoria (primo, secondo, dessert). Per ogni categoria costruiamo un grafo di *flusso massimo* che assegna piatti agli studenti rispettando (i) le preferenze e (ii) le limitazioni di capacità (numero di porzioni disponibili). Il numero massimo di assegnazioni studente–piatto si ottiene sommando i tre massimi flussi di categoria.

Costruzione del grafo per una categoria fissata c . Sia $P_c = \{p \in P : t_p = c\}$ l'insieme dei piatti della categoria c . Costruiamo un grafo orientato $G_c = (V_c, E_c)$ con:

- un nodo sorgente s e un nodo pozzo t ;
- un nodo per ogni studente $s_i \in S$;
- un nodo per ogni piatto $p \in P_c$.

Archi e capacità:

- da s a ogni studente s_i : capacità 1 (ogni studente può ricevere al più un piatto della categoria c);
- da s_i a $p \in P_c$: capacità 1 se $p \in A_{s_i}$ (rispetto delle preferenze);
- da ogni piatto $p \in P_c$ a t : capacità m_p (numero di porzioni disponibili del piatto p).

Calcolo del flusso e soluzione complessiva. Eseguiamo un algoritmo di flusso massimo su G_c e otteniamo un valore F_c . Ripetiamo la procedura per $c = \text{primo}, \text{secondo}, \text{dessert}$.

Il numero massimo di assegnazioni studente–piatto è

$$F_{tot} = F_{primo} + F_{secondo} + F_{dessert}.$$

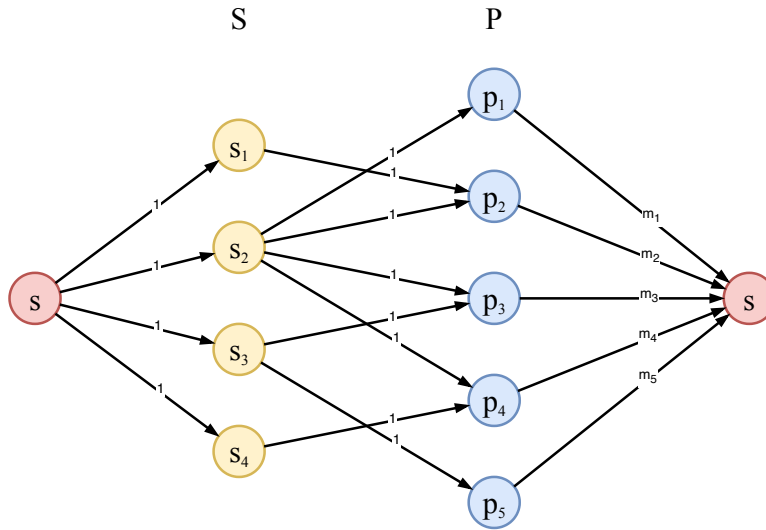


Figura 1: Rete di flusso

Complessità. Sia $n = |S|$, $m_c = |P_c|$, ed $E_c = |\{(s_i, p) : p \in P_c \cap A_{s_i}\}|$. Il grafo G_c ha $V_c = n + m_c + 2$ nodi e $|E_c| + n + m_c$ archi. La complessità per ogni categoria è $O((V_c + E_c) \cdot n)$. Ripetendo per tre categorie, la complessità totale è la somma delle tre istanze.

La figura 1 mostra l'organizzazione di ciascuna delle reti.

Esercizio B2 – Sottoinsiemi k -distanti – Punti ≥ 10

La soluzione più semplice consiste nel generare tutti i 2^n sottoinsiemi di un vettore A e, prima di stamparne uno, verificare che rispetti le regole dei sottoinsiemi k -distanti; in caso affermativo, lo si stampa. Se la verifica è implementata ingenuamente controllando tutte le coppie di elementi del sottoinsieme corrente (costo $O(|S|^2) \leq O(n^2)$), il costo complessivo è

$$\Theta(n^2 \cdot 2^n),$$

dove usiamo Θ perché non si fa alcun pruning: si visitano e si verificano tutti i sottoinsiemi.

Una soluzione più elegante ordina preventivamente A e rimuove le occorrenze duplicate, così da lavorare su m valori distinti ($m \leq n$). Costruendo i sottoinsiemi in ordine crescente, per aggiungere un nuovo valore v è sufficiente confrontarlo con l'ultimo valore inserito u : si può aggiungere v se e solo se $|v - u| > k$. In questo modo si evita una verifica quadratica sul sottoinsieme. Il costo diventa

$$O(n \log n) \text{ (ordinamento)} + O(2^m) \text{ (esplorazione)} + O(n \cdot 2^m) \text{ (stampa)}.$$

Poiché la stampa di tutti gli insiemi prodotti richiede nel peggiore dei casi $O(n \cdot 2^m)$ caratteri e $m \leq n$, tale termine domina: in sintesi, il costo totale è $O(n \cdot 2^n)$.

Il codice è il seguente:

```
printDistantSets(int[] A, int n, int k)
    sort(A, n)
    int m = 0                                % Nuova dimensione dopo aver rimosso i duplicati
    for i = 1 to n do
        if m = 0 or A[i - 1] != A[i] then
            m = m + 1
            A[m] = A[i]
    STACK S = new Stack
    printRec(A, m, k, S)
```

```

printRec(int[] A, int i, int k, STACK S)
    if i==0 then
        print S
    else
        if S.isEmpty() or S.top() - A[i] > k then
            S.push(A[i])
            printRec(A, i - 1, k, S)
            S.pop()

```

Esercizio B3 – Going nowhere – Punti ≥ 12

Come prima soluzione, consideriamo una matrice $DP[0 \dots n][-\lfloor n/2 \rfloor \dots \lfloor n/2 \rfloor]$, dove $DP[steps][pos]$ rappresenta il numero di modi per raggiungere la posizione 0 a partire dalla posizione pos utilizzando esattamente $steps$ mosse. Il valore $\lfloor n/2 \rfloor$ rappresenta la distanza massima da 0 che può essere necessaria per una soluzione, poiché ogni mossa verso destra (R) deve essere bilanciata da una mossa verso sinistra (L) per tornare a zero.

La matrice DP può essere calcolata tramite la seguente formula ricorsiva:

$$DP[steps][pos] = \begin{cases} 0 & |pos| > steps \\ 1 & steps = 0 \text{ and } pos = 0 \\ DP[steps-1, pos-1] + DP[steps-1, pos] + DP[steps-1, pos+1] & \text{altrimenti} \end{cases}$$

La formula può essere interpretata nel seguente modo:

- Se la distanza da 0 è maggiore del numero di passi disponibili, non è possibile tornare alla posizione iniziale: il risultato è 0. Questo caso copre anche la situazione in cui $steps = 0$ e $pos \neq 0$.
- Se ci troviamo nella posizione 0 e non abbiamo più passi a disposizione, abbiamo trovato una sequenza valida: il risultato è 1.
- Negli altri casi, il numero totale di soluzioni si ottiene sommando le possibilità che derivano da una mossa verso sinistra, una mossa senza movimento, o una mossa verso destra, decrementando di uno il numero di passi rimanenti.

Risolviamo il problema tramite memoization, evitando di costruire esplicitamente tutta la matrice DP e calcolando solo i valori necessari.

```

int countWays(int n)
    int[] DP[1 .. n][ -⌊n/2⌋ .. ⌊n/2⌋ ] = {-1}
    return cwRec(DP, n, 0)

```

```

int cwRec(int[][] DP, int steps, int pos)
    if |pos| > steps then
        return 0
    if steps == 0 and pos == 0 then
        return 1
    if DP[steps][pos] == -1 then
        DP[steps][pos] = cwRec(DP, steps - 1, pos - 1) + cwRec(DP, steps - 1, pos) + cwRec(DP, steps - 1, pos + 1)
    return DP[pos][steps]

```

La complessità temporale dell'algoritmo è $\Theta(n^2)$, dato che per ciascuno dei n passi consideriamo al più $2 \cdot \lfloor n/2 \rfloor + 1 = \Theta(n)$ posizioni diverse. Lo spazio utilizzato per memorizzare i risultati parziali è anch'esso $\Theta(n^2)$.

Approfondimenti, o come perdere una domenica pomeriggio La sequenza generata corrisponde a <https://oeis.org/A002426>. Leggendo la descrizione, scopriamo che corrisponde al numero di Grand Motzkin Path di dimensione n .

Un *Motzkin Path* di lunghezza n è un cammino sul reticolo che parte dal punto $(0,0)$ e arriva al punto $(n,0)$ utilizzando solo i seguenti tipi di passo:

- passo *ascendente diagonale* $(1, +1)$,

- passo *orizzontale* $(1, 0)$,
- passo *discendente diagonale* $(1, -1)$.

con la condizione che il cammino *non scenda mai sotto l'asse x* , ovvero tutti i punti del cammino devono avere coordinata verticale maggiore o uguale a zero.

Un *Grand Motzkin Path* è simile, ma rimuove il vincolo sull'altezza: si tratta sempre di un cammino da $(0, 0)$ a $(n, 0)$ con gli stessi tipi di passo, ma il cammino può scendere anche *sotto* l'asse delle ascisse. La versione presentata elimina il movimento orizzontale, ma il principio è lo stesso.

Ricapitolando:

<i>Caratteristica</i>	<i>Motzkin Path</i>	<i>Grand Motzkin Path</i>
Tipo di passi	$(1, 1), (1, 0), (1, -1)$	$(1, 1), (1, 0), (1, -1)$
Punto iniziale	$(0, 0)$	$(0, 0)$
Punto finale	$(n, 0)$	$(n, 0)$
Può scendere sotto l'asse x ?	No	Sì
Vincolo di altezza	$y \geq 0$	Nessun vincolo

La Figura 2 seguente mostra i 19 Grand Motzkin Path per $n = 4$:

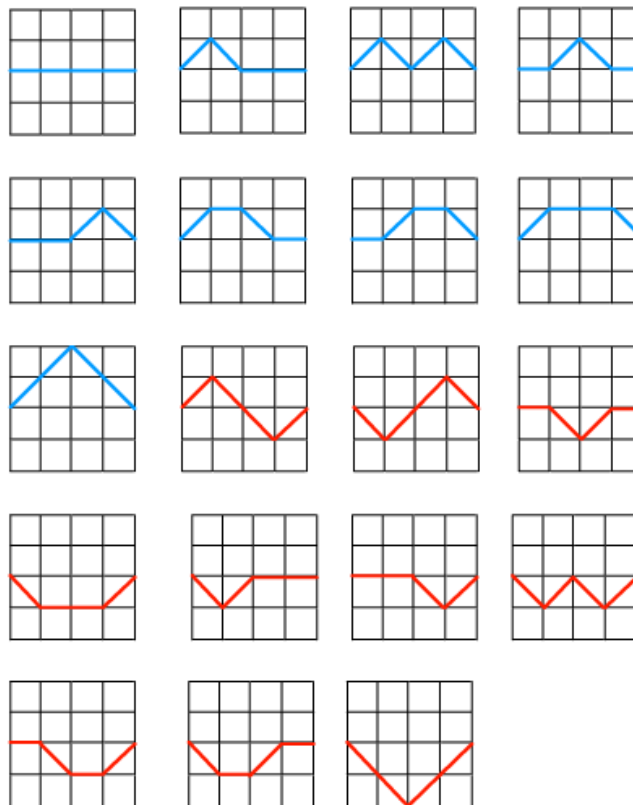


Figura 2: I 19 Grand Motzkin Path per $n = 4$; i primi 9 sono anche Motzkin Path, perchè non scendono sotto l'asse dell'ascisse. Fonte: <https://epubs.siam.org/doi/10.1137/080723090>

È interessante osservare come i Grand Motzkin Paths compaiano in un contesto apparentemente molto distante, ovvero nell'analisi delle latenze introdotte dalla sincronizzazione locale in sistemi di calcolo parallelo. In particolare, nell'articolo "Analysis of Delays Caused by Local Synchronization" (Lipman & Stout, SIAM Journal on Computing, 2010), viene mostrato che il numero di stati del sistema (modellato tramite una catena di Markov) è uguale al numero di Grand Motzkin Path di una certa lunghezza. Questo collegamento tra strutture combinatorie classiche e prestazioni di sistemi paralleli asincroni è un bel esempio di come idee provenienti dalla matematica discreta trovino applicazioni in ambiti computazionali sofisticati.